

SRN.No.	31240250
Name:	Shaikh Mohammed Anas

VISHWAKARMA UNIVERSITY, PUNE
University Grants Commission (UGC) Approved State Private University

Examination: Project-1			
Academic Year	2025-2026	Pattern	2025-2026
Faculty	Commerce	Course Name	DevOps
Department	Management	Course Code	MCA24403
Programme	MCA	Semester	IV
Division	A	Course Teacher (s)	Dr. Bharati Kawade
Maximum Marks	25	Total No. of Pages	6
Project Start Date	15/03/2026	Project End Date	18/04/2026
Instructions to Candidates: - - Students need to strictly adhere to the project deadlines - Project content should be unique and plagiarism is strictly prohibited. - Submission of project report in a prescribed format is mandatory.			

Project Title: FACE RECOGNITION BASED ATTENDANCE SYSTEM

Project Objective:

The Face Recognition Based Attendance System automates student attendance using real-time face recognition. The system uses OpenCV's Haar Cascade algorithm for face detection and the LBPH (Local Binary Pattern Histograms) algorithm for recognition. Students are registered by capturing 100 face images, a model is trained using these images, and attendance is marked automatically when a registered face is detected by the camera. The project is containerized using Docker and the Docker image is deployed to AWS ECR (Elastic Container Registry) demonstrating a complete DevOps pipeline.

Project Outcome:

- Students can register their face by entering ID and Name and clicking Take Images — 100 grayscale face photos are captured and saved automatically.
- The LBPH model is trained on all registered face images by clicking Save Profile — the trained model is saved as Trainner.yml.
- Attendance is marked automatically when a registered face is recognised by the camera with confidence score below 50.
- Daily attendance records are saved as CSV files with student ID, name, date and timestamp.
- The project is containerized using Docker and the image is pushed to AWS ECR for cloud deployment.

Project Description	
1.	Problem Definition / Identification Traditional attendance systems rely on manual roll calls which are time-consuming, error-prone, and vulnerable to proxy attendance. There is no reliable way to verify student identity automatically. This project addresses this problem by implementing a face recognition based system that automatically identifies registered students and marks their attendance in real time without any manual intervention. The system also demonstrates DevOps practices including version control with GitHub, containerization with Docker, and cloud deployment with AWS ECR.

2.

Methodology

Stage 1 — Face Registration: Student enters ID and Name. Camera opens and Haar Cascade detects face in each frame. 100 grayscale images saved to TrainingImage/ folder. Student details saved to StudentDetails.csv.

Stage 2 — Model Training: LBPH algorithm reads all saved face images. Extracts texture fingerprint (Local Binary Pattern Histograms) from each image. Trained model saved as TrainingImageLabel/Trainer.yml.

Stage 3 — Attendance Marking: Camera opens and loads trained model. Haar Cascade detects face. recognizer.predict() returns serial number and confidence score. If confidence < 50, attendance marked with timestamp in daily CSV file.

DevOps Pipeline: Code pushed to GitHub for version control. Dockerfile created for containerization. Docker image built on AWS CloudShell. Image tagged and pushed to AWS ECR private registry.

3.

Implementation

Algorithms Used:

Haar Cascade — Pre-trained XML model by Viola-Jones. Scans every video frame at multiple scales for face-shaped light/shadow patterns. Used for face detection in both registration and attendance stages.

LBPH (Local Binary Pattern Histograms) — For each pixel, compares brightness to 8 neighbours and writes binary code. Creates texture fingerprint histogram. Matches live face against stored fingerprints using confidence score (lower = better match, threshold = 50).

Key Functions:

Function	Description
TakeImages()	Opens camera, captures 100 face images, saves to TrainingImage/, updates StudentDetails.csv
TrainImages()	Loads all face images, trains LBPH model, saves Trainer.yml
TrackImages()	Opens camera in background thread, recognises faces, marks attendance, updates GUI table
fix_student_details()	Auto-removes duplicate CSV entries and reassigns clean serial numbers on startup
refresh_table()	Updates attendance treeview table using pandas after camera closes

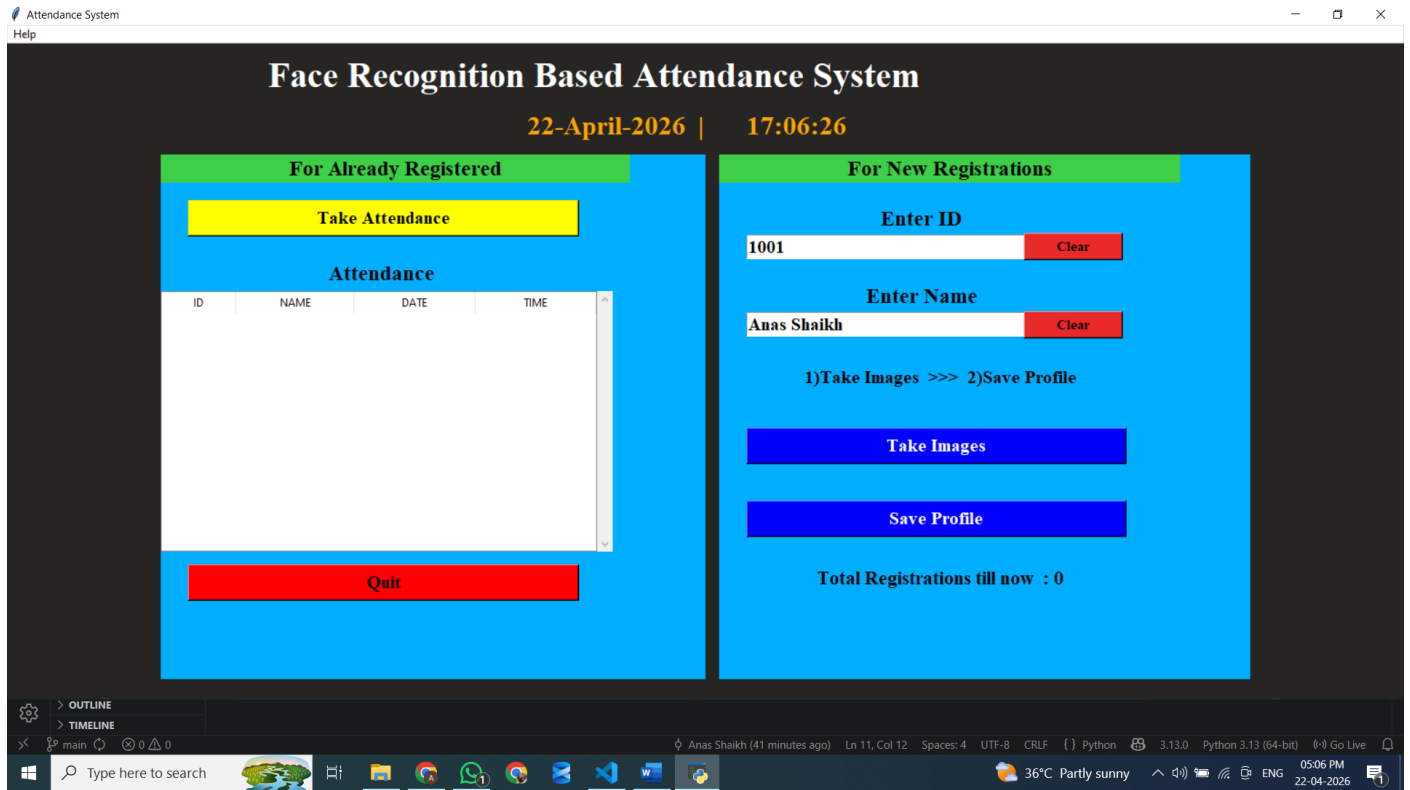
DevOps Implementation:

Step	Tool / Command	Result
Version Control	git init, git push → GitHub	github.com/shaikhanas78/face-attendance-system
Containerization	Dockerfile + requirements.txt	Docker image defined
Build Image	docker build -t face-attendance-system .	Built in 138 seconds on AWS
Cloud Login	aws ecr get-login-password docker login	Login Succeeded
Tag Image	docker tag face-attendance-system:latest ...	Image tagged with ECR URI
Push to Cloud	docker push .../face-attendance-system:latest	Image live on AWS ECR

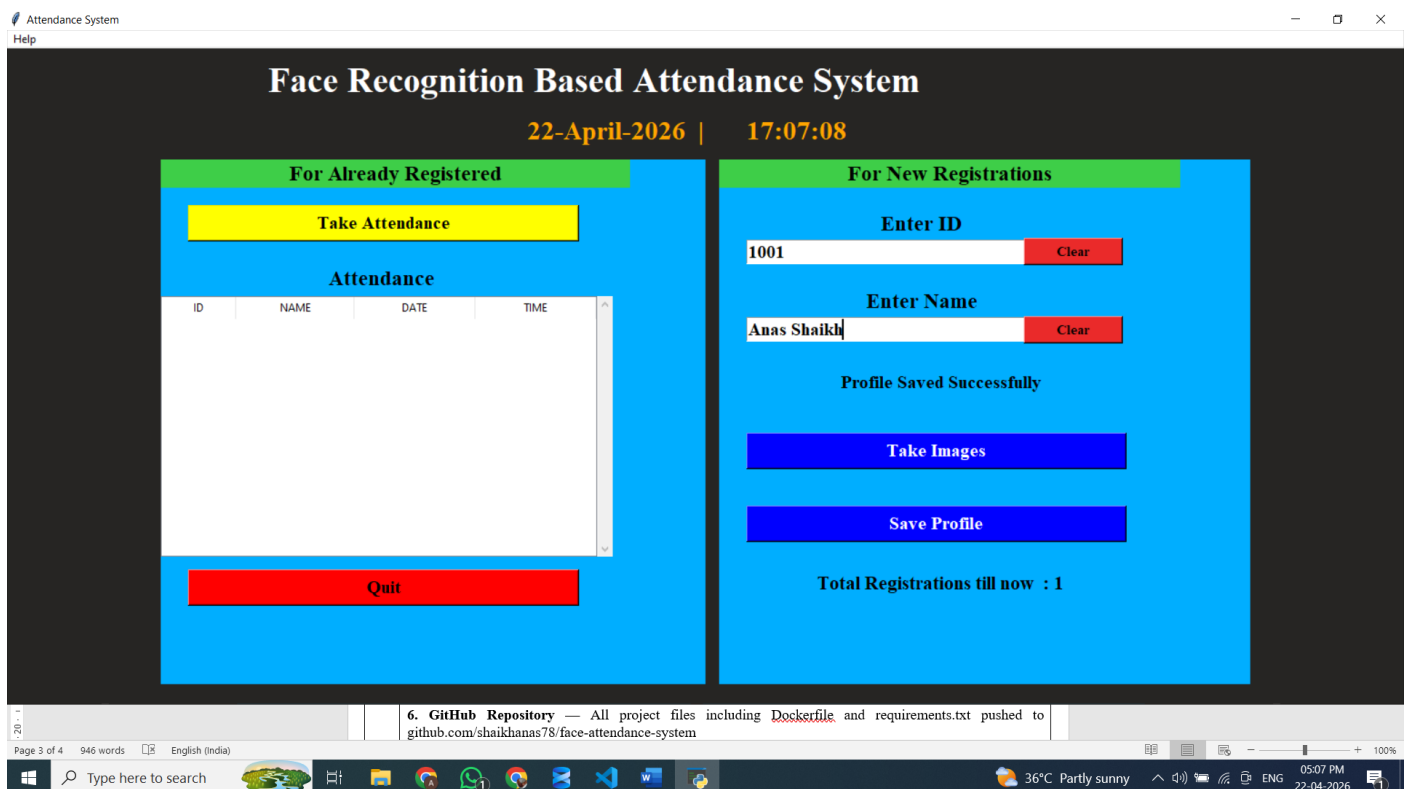
4.

Output & Evidence:

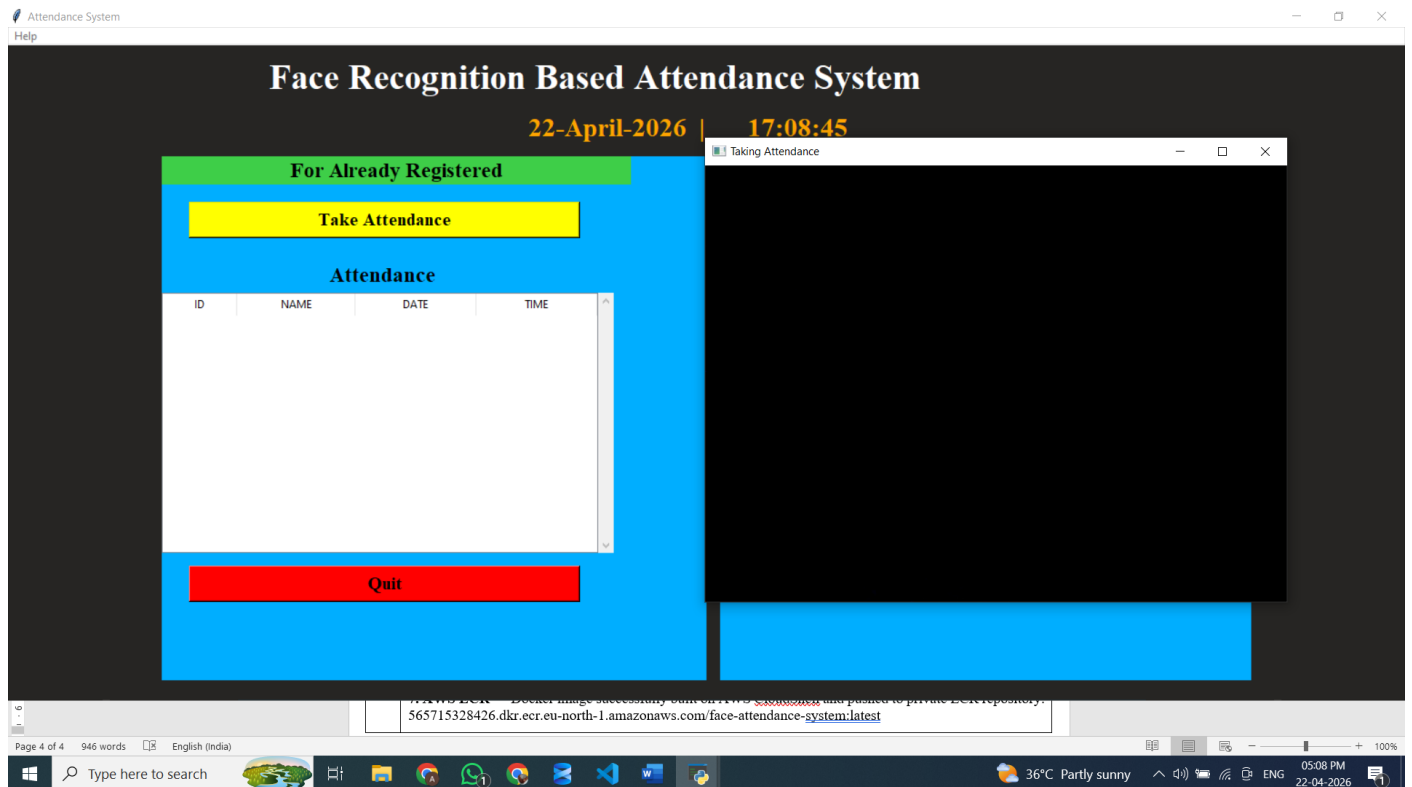
1. Face Registration — Student enters ID and Name. Camera opens and captures 100 face images. StudentDetails.csv updated with serial number, ID and name.



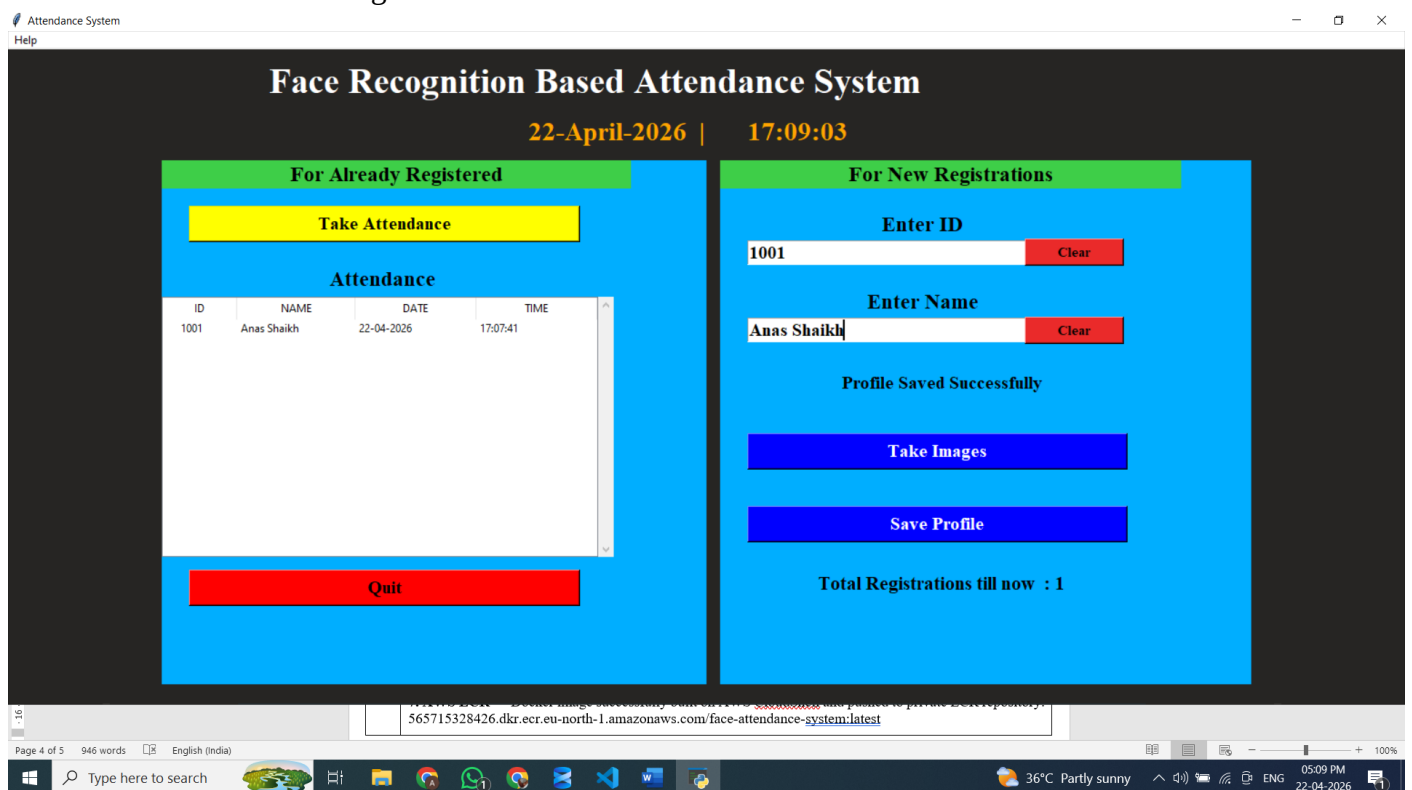
2. Model Training — Click Save Profile, enter password. LBPH model trained on all face images. Message displays 'Profile Saved Successfully' and Trainer.yml is created in TrainingImageLabel/ folder.



3. Attendance Marking — Camera opens, detects face in real time with blue rectangle. Recognised name displayed on screen. Press Q to close camera.



4. Attendance Table — After pressing Q, attendance table in GUI updates automatically showing ID, Name, Date and Time for each recognised student.



5. CSV Output — Attendance_DD-MM-YYYY.csv created in Attendance/ folder with columns: Id, Name, Date, Time for every session.

Attendance_22-04-2026.csv - Excel

File Home Insert Draw Page Layout Formulas Data Review View Help Acrobat Tell me what you want to do

Clipboard Font Alignment Number

Conditional Formatting Format as Table Cell Styles Insert Delete Format Cells Sort & Filter Find & Select Editing Create PDF and Share link Share via Outlook Adobe Acrobat

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	Id	Name	Date			Time																	
2	1001	Anas Shaikh	22-04-2026			17:07:41																	
3																							
4																							
5																							
6																							
7																							
8																							
9																							
10																							
11																							
12																							
13																							
14																							
15																							
16																							
17																							
18																							
19																							
20																							
21																							
22																							
23																							
24																							
25																							
26																							
27																							
28																							
29																							

Attendance_22-04-2026

Ready 36°C Partly sunny 05:09 PM 22-04-2026

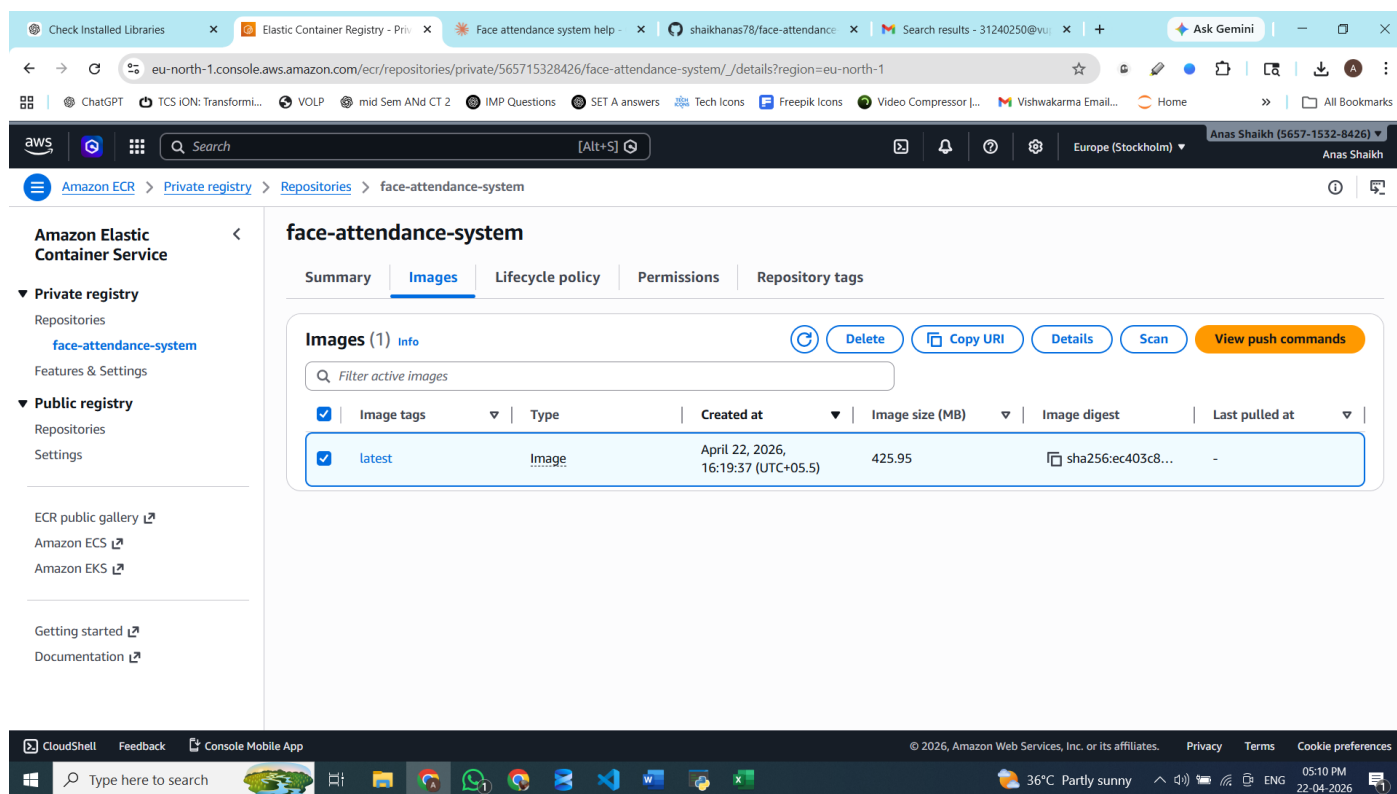
6. GitHub Repository — All project files including Dockerfile and requirements.txt pushed to github.com/shaikhanas78/face-attendance-system

The screenshot displays the GitHub interface for the repository 'face-attendance-system'. The repository is owned by 'shaikhanas78' and is public. It has 0 watches, 0 forks, and 0 stars. The file list shows the following files, all committed 1 hour ago:

- .dockerignore
- .gitignore
- Dockerfile
- README.md
- haarcascade_frontalface_default.xml
- main.py
- requirements.txt

The README section is partially visible at the bottom, showing the title 'Face Recognition Based Attendance System'.

7. AWS ECR — Docker image successfully built on AWS CloudShell and pushed to private ECR repository: 565715328426.dkr.ecr.eu-north-1.amazonaws.com/face-attendance-system:latest



CO: Course Outcome, BTL: Bloom's Taxonomy Level

Course Outcomes

CO No.	Statement
CO1	Understand DevOps concepts and containerization principles (Understand)
CO2	Apply version control using Git and GitHub (Apply)
CO3	Implement Docker containerization for Python applications (Apply)
CO4	Deploy Docker images to AWS cloud services (Apply)
CO5	Demonstrate a complete DevOps pipeline from code to cloud (Create)

Bloom's Taxonomy Level (BTL)

BTL No.	BTL	Statement
1	Remember	Recall facts and basic concepts
2	Understand	Explain ideas or concepts
3	Apply	Use information in new situations
4	Analyze	Draw connections among ideas
5	Evaluate	Justify a stand or decision
6	Create	Produce new or original work